



Universidad
Nacional
de Córdoba



Facultad de
Ciencias Exactas
Físicas y Naturales

Fundación Fulgor

Diseño Digital Avanzado

GP01

Nombre	DNI
Corvalán, Abel Nicolás	41.220.050

Docentes	Ing. Pola, Ariel Cayuela, Pablo.
----------	-------------------------------------

Córdoba, República Argentina
9 de septiembre de 2026

Índice

1.	4
2. Ejercicio 1	4
2.1. Análisis de etapas	4
2.2. Consignas	4
2.2.1. adder_4b	4
2.2.2. adder_7b	5
2.2.3. multiplexer	5
2.2.4. register1	5
2.2.5. top	6
3. Ejercicio 2	9
4. Ejercicio 3	10
5. Ejercicio 4	11
5.0.1. Camino directo (términos de entrada)	14
5.0.2. Camino de realimentación	15
5.0.3. Cálculo del peor caso total	15
6. LABORATORIO 2 - FEC - Grupo 1	15
6.1. Ejercicio 1 - Matrices y parámetros	15
6.2. Ejercicio 2 - Distancia mínima y capacidad de corrección	16

Índice de figuras

1.	Esquemático del sistema	4
2.	Diagrama de onda, overflow en sistema	9
3.	Diagrama de onda, sistema de operaciones	9
4.	Diagrama de onda, sistema de operaciones hexadecimal	10
5.	Diagrama de onda, filtro IIR	14

Índice de tablas

1.

2. Ejercicio 1

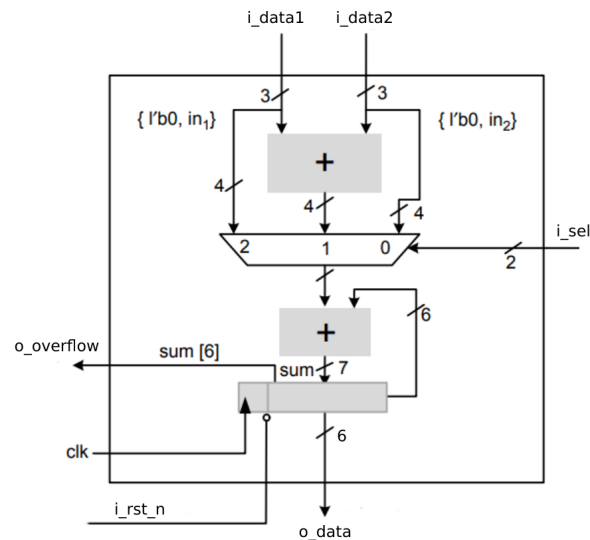


Figura 1: Esquemático del sistema

2.1. Análisis de etapas

2.2. Consignas

1. Escribir el código Verilog para implementar el diseño de la Figura 1 considerando reset asíncrono. Se modela el sistema en 4 módulos diferentes:

- adder_4b.
- adder_7b.
- multiplexer.
- register1.

A continuación se muestra el código verilog de cada etapa:

2.2.1. adder_4b

```
1 module adder_4b(  
2     // Outputs.  
3     output wire [3:0] o_sum1,  
4     // Inputs.  
5     input wire [2:0] i_data1,  
6     input wire [2:0] i_data2  
7 );  
8  
9 assign o_sum1 = i_data1 + i_data2;  
10  
11 endmodule
```

2.2.2. adder_7b

```
1 module adder_7b(  
2     // Output.  
3     output [6:0] o_sum2,  
4     // Inputs.  
5     input  [5:0] i_sumA,  
6     input  [5:0] i_sumB  
7 );  
8 assign o_sum2 = i_sumA + i_sumB;  
9 endmodule
```

2.2.3. multiplexer

```
1 module multiplexer(  
2     // Outputs.  
3     output reg [3:0] o_multiplexer,  
4     // Inputs.  
5     input          [3:0] i_0      ,  
6     input          [3:0] i_1      ,  
7     input          [3:0] i_2      ,  
8     input          [1:0] i_sel  
9 );  
10  
11 always@(*) begin  
12     case (i_sel)  
13         2'b00: o_multiplexer = i_0;  
14         2'b01: o_multiplexer = i_1;  
15         2'b10: o_multiplexer = i_2;  
16         default: o_multiplexer = 4'b0000;  
17     endcase  
18 end  
19 endmodule
```

2.2.4. register1

```
1 module register1(  
2  
3     // Output.  
4     output reg [5:0] o_data      , // Output  
5     output reg      o_overflow, // Overflow  
6     // Inputs.  
7     input          [6:0] i_sum      , // Input  
8     input          clk          , // Clock Signal  
9     input          i_rst_n        // Negative Reset Signal  
10 );  
11  
12 always @(posedge clk or i_rst_n) begin  
13     if(!i_rst_n) begin  
14         o_data      = 6'b000000 ;  
15         o_overflow = 1'b0      ;  
16     end else begin  
17         //o_data      <= i_sum[5:0];  
18         //o_overflow <= i_sum[6] ;  
19         {o_overflow, o_data} = i_sum;  
20     end  
21 end  
22 endmodule
```

2.2.5. top

```
1 module top(
2     // Outputs.
3     output [5:0] o_data    ,
4     output      o_overflow,
5     // Inputs.
6     input  [2:0] i_data1    ,
7     input  [2:0] i_data2    ,
8     input  [1:0] i_sel      ,
9     input      clk          ,
10    input      i_rst_n
11 );
12
13 adder_4b
14     adder_4b(
15         .o_sum1    (o_sum1) ,
16
17         .i_data1    (i_data1),
18         .i_data2    (i_data2)
19     );
20
21 wire [3:0] i_0;
22 wire [3:0] o_sum1;
23 wire [3:0] i_2;
24
25 assign i_0 = {1'b0, i_data2};
26 assign i_2 = {1'b0, i_data1};
27
28 multiplexer
29     multiplexer(
30         .o_multiplexer(o_multiplexer),
31
32         .i_0            (i_0)    ,
33         .i_1            (o_sum1),
34         .i_2            (i_2)    ,
35         .i_sel          (i_sel)
36     );
37
38 wire [3:0] o_multiplexer;
39 wire [5:0] i_sum2 = {2'b00, o_multiplexer};
40
41 adder_7b
42     adder_7b(
43         .o_sum2            (o_sum2)    ,
44         .i_sumA            (i_sum2)    ,
45         .i_sumB            (o_data)    // connect with output of register1.
46     );
47
48 wire [6:0] o_sum2;
49
50 register1
51     register1(
52         .o_data            (o_data)    ,
53         .o_overflow        (o_overflow),
54
55         .i_sum             (o_sum2)    ,
56         .clk               (clk)       ,
57         .i_rst_n           (i_rst_n)
58     );
59 endmodule
```

2. Genere la señal de reset apropiada para el registro de realimentación utilizado en el diseño.

En el módulo *register1.v* se diseña el reset asíncrono del sistema. A continuación se muestra el código verilog con el bloque procedural que contiene la señal de reset *i_rst_n*.

```

1 always @(posedge clk or i_rst_n) begin
2     if(!i_rst_n) begin
3         o_data      = 6'b000000 ;
4         o_overflow = 1'b0      ;
5     end else begin
6         //o_data      <= i_sum[5:0];
7         //o_overflow <= i_sum[6]  ;
8         {o_overflow, o_data} = i_sum;
9     end
10 end

```

3. Escribir un testbench que permita verificar el correcto funcionamiento del circuito propuesto. Los estímulos pueden ser generados con python o modelados en el testbench.

Se genera el siguiente código test bench para verificar el correcto funcionamiento del sistema propuesto:

```

1 `timescale 1ns/1ps
2
3 module tb_top();
4
5 wire [5:0] o_data      ;
6 wire      o_overflow;
7
8 reg  [2:0] i_data1     ;
9 reg  [2:0] i_data2     ;
10
11 reg  [1:0] i_sel       ;
12 reg      clk          ;
13 reg      i_rst_n      ;
14
15 top
16     top(
17         .o_data      (o_data)      ,
18         .o_overflow  (o_overflow),
19         .i_data1     (i_data1)     ,
20         .i_data2     (i_data2)     ,
21         .i_sel       (i_sel)       ,
22         .clk         (clk)         ,
23         .i_rst_n     (i_rst_n)
24     );
25
26 initial begin
27     clk      = 1'b0;
28     i_data1  = 3'b000;
29     i_data2  = 3'b000;
30     i_sel    = 2'd0;
31     i_rst_n  = 1'b1;
32 end
33
34 always begin
35     #5 clk = ~clk;
36 end
37
38 initial begin
39     #10 i_data1 = 3'd2; i_data2 = 3'd2;
40     #10 i_data1 = 3'd1; i_data2 = 3'd1;
41     #2  i_rst_n = 1'b0;
42     #10 i_data1 = 3'd5; i_data2 = 3'd5;
43     #10 i_data1 = 3'd5; i_data2 = 3'd5;
44     #4  i_rst_n = 1'b1;

```

```

45     #10 i_sel    = 2'd1;
46     #5  i_data1 = 3'd1; i_data2 = 3'd1;
47     #30;
48     $finish;
49 end
50 endmodule

```

4. ¿Cuántos ciclos de reloj son necesarios para que el registro *o_data* produzca overflow cuando *i_sel*, *i_data1* e *i_data2* son iguales a 1?

Para obtener la cantidad de ciclos necesarios para que el registro produzca overflow se escribe el siguiente código test bench de descripción de hardware en lenguaje Verilog.

```

1  'timescale 1ns/1ps
2
3  module tb_top();
4  wire [5:0] o_data      ;
5  wire      o_overflow;
6
7  reg  [2:0] i_data1     ;
8  reg  [2:0] i_data2     ;
9
10 reg  [1:0] i_sel       ;
11 reg      clk           ;
12 reg      i_rst_n       ;
13
14 top
15   top(
16       .o_data      (o_data)      ,
17       .o_overflow   (o_overflow),
18       .i_data1      (i_data1)     ,
19       .i_data2      (i_data2)     ,
20       .i_sel        (i_sel)       ,
21       .clk          (clk)         ,
22       .i_rst_n      (i_rst_n)    ,
23   );
24
25 initial begin
26     clk      = 1'b0;
27     i_data1  = 3'b000;
28     i_data2  = 3'b000;
29     i_sel    = 2'd0;
30     i_rst_n  = 1'b1;
31 end
32
33 always begin
34     #5 clk = ~clk;
35 end
36
37 initial begin
38     #10 i_data1 = 3'd2; i_data2 = 3'd2;
39     #10 i_data1 = 3'd1; i_data2 = 3'd1;
40     #2  i_rst_n = 1'b0;
41
42     #10 i_data1 = 3'd5; i_data2 = 3'd5;
43     #10 i_data1 = 3'd5; i_data2 = 3'd5;
44
45     #4  i_rst_n = 1'b1;
46     #10 i_sel   = 2'd1;
47     #5  i_data1 = 3'd1; i_data2 = 3'd1;
48     #30;
49     $finish;

```

```

50 end
51 endmodule

```

Se obtiene el siguiente diagrama de onda para el test bench con las señales pedidas:

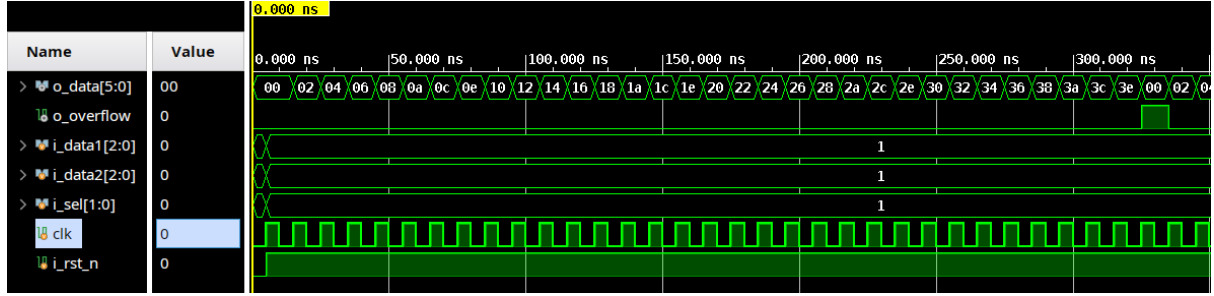


Figura 2: Diagrama de onda, overflow en sistema

Se tiene que un periodo de la señal de clock clk es igual a 10 ns . Hay 32 ciclos hasta que se produzca el overflow con las condiciones $i_sel = 1$, $i_data1 = 1$, $i_data2 = 1$.

$$Ciclos_{overflow} \frac{t_{overflow}}{T_{ciclo}} = \frac{320\text{ ns}}{10\text{ ns}} = 32\text{ ciclos}$$

$$Ciclos_{overflow} = 32\text{ ciclos}$$

3. Ejercicio 2

1. Realizar el esquemático o diagrama en bloque del datapath de un selector de operaciones que ejecuta las siguientes operaciones aritméticas en paralelo con dos entradas i_dataA e i_dataB de tipo signadas de 16 bits y asigne el valor del resultado a una salida o_dataC de 16 bits .
2. La elección de la operación a realizar depende de una señal de control i_sel de 2 bits .
3. Implementar el diseño en Verilog y el testbench para verificar el comportamiento.

Operaciones:

$$o_dataC = i_dataA + i_dataB$$

$$o_dataC = i_dataA - i_dataB$$

$$o_dataC = i_dataA \& i_dataB$$

$$o_dataC = i_dataA | i_dataB$$

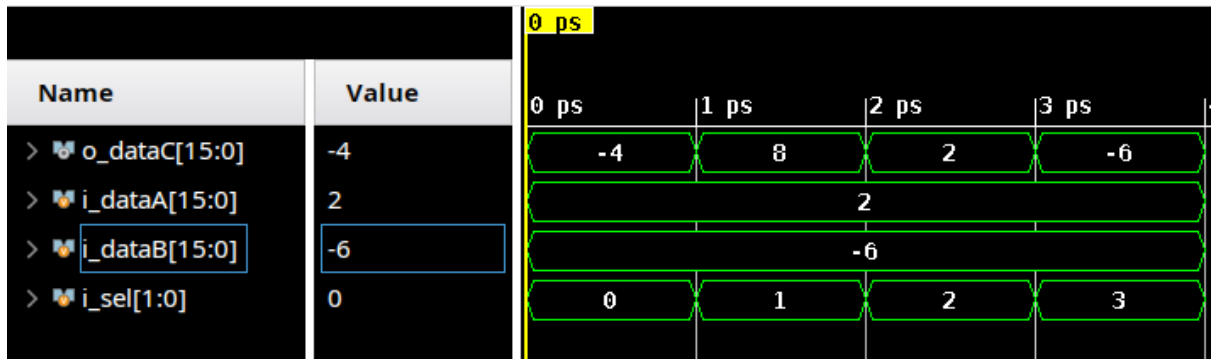


Figura 3: Diagrama de onda, sistema de operaciones

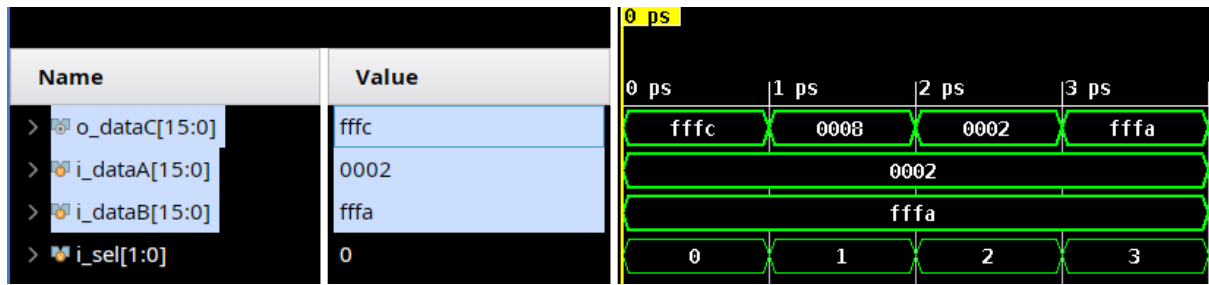


Figura 4: Diagrama de onda, sistema de operaciones hexadecimal

4. Ejercicio 3

Dibuje el esquemático para el siguiente código Verilog. Especifique de forma clara los tamaños de datos para todos los cables y muestre multiplexores, registros, señales de clock y reset.

```

1 module test_module (
2     input      [31:0]    x0,
3     input      [1:0]     sel,
4     input      clk,
5     input      rst_n,
6     output reg  [31:0]   y0
7 );
8
9 reg  [31:0] x1, x2, x3;
10 reg  [31:0] y1;
11 wire [31:0] out;
12
13 assign out = (x0 + x1 + x2 + x3 + y1) >>> 2;
14
15 always @(posedge clk or negedge rst_n) begin
16     if (!rst_n) begin
17         x1 <= 0;
18         x2 <= 0;
19         x3 <= 0;
20     end
21     else if (sel == 0) begin
22         x3 <= x2;
23         x2 <= x1;
24         x1 <= x0;
25     end
26     else if (sel == 01) begin
27         x3 <= x1;
28         x2 <= x0;
29         x1 <= x2;
30     end
31     else begin
32         x3 <= x3;
33         x2 <= x2;
34         x1 <= x0;
35     end
36 end
37
38 always @(posedge clk or negedge rst_n) begin
39     if (!rst_n) begin
40         y1 <= 0;
41         y0 <= 0;
42     end
43     else begin
44         y1 <= y0;
45         y0 <= out;

```

```

46     end
47 end
48 endmodule

```

Este código modela y describe un filtro promedio (moving average) con realimentación. Es un sistema IIR (Infinite Impulse Response) a causa de la realimentación $y[n-1]$. No depende solo de las entradas pasadas.

5. Ejercicio 4

$$y[n] = x[n] - x[n-1] + x[n-2] + x[n-3] + 0,5y[n-1] + 0,25y[n-2]$$

- Dibujar la arquitectura de la siguiente ecuación diferencial.
- Escribir el código Verilog.

Código de descripción del filtro IIR con entrada de 8 bits (incluyendo signo).

```

1  module top(
2      // Output
3      output reg signed [10:0] y ,
4      // Delayed inputs
5      output reg signed [7:0] x1,
6      output reg signed [7:0] x2,
7      output reg signed [7:0] x3,
8      // Outputs for debug
9      output reg signed [10:0] y1,
10     output reg signed [10:0] y2,
11     // Input
12     input signed [7:0] x,
13     input clk,
14     input rst_n
15 );
16
17 // We need 11 bits to representate
18 // -512 to 512
19 wire signed [10:0] y_o;
20
21 // Equation implementation
22 assign y_o = x - x1 + x2 + x3 + (y1 >>> 1) + (y2 >>> 2);
23
24 always @(posedge clk or negedge rst_n) begin
25     if(!rst_n) begin
26         x1 <= 0;
27         x2 <= 0;
28         x3 <= 0;
29         y <= 0;
30         y1 <= 0;
31         y2 <= 0;
32     end else begin
33         // Shift registers for input
34         x1 <= x;
35         x2 <= x1;
36         x3 <= x2;
37
38         // Output
39         y <= y_o;
40
41         // Feedback paths
42         y1 <= y_o;
43         y2 <= y1;
44     end

```

```
45 end
46
47 endmodule
```

Código de testbench para el sistema.

```
1  'timescale 1ns/1ps
2
3  module tb_top();
4
5  wire signed [10:0] out;
6
7  wire signed [7:0] x1;
8  wire signed [7:0] x2;
9  wire signed [7:0] x3;
10
11 wire signed [10:0] y1;
12 wire signed [10:0] y2;
13
14 reg signed [7:0] x;
15 reg             clk;
16 reg             rst_n;
17
18 always #5 clk = ~clk;
19
20 initial begin
21     clk = 0;
22     rst_n = 0;
23     x = -8'sd128;
24
25     #20 rst_n = 1;
26
27     // Repeat this block 256 times.
28     // Control event on x.
29     // This block increment the value of x.
30     repeat(256) begin
31         @(posedge clk);
32         x = x + 1;
33     end
34     #100 $stop;
35 end
36 top
37     top(
38         .y(out),
39         .x1(x1),
40         .x2(x2),
41         .x3(x3),
42         .y1(y1),
43         .y2(y2),
44         .x(x),
45         .clk(clk),
46         .rst_n(rst_n)
47     );
48 endmodule
```

A continuación se muestra el diagrama de onda del filtro IIR

- Considerar valores enteros en las muestras de entrada.
- Considerar 8bits para la representación de x y calcular el número de bits para la salida considerando full-resolution.

Se consideran 8 bits con signo para la entrada $x[n]$ en la salida se debería tener 11 bits, esto se debe a que:

El valor máximo y mínimo que se puede tener en la entrada por los siguientes cálculos:

$$X_{max} = 2^{b-1} - 1$$

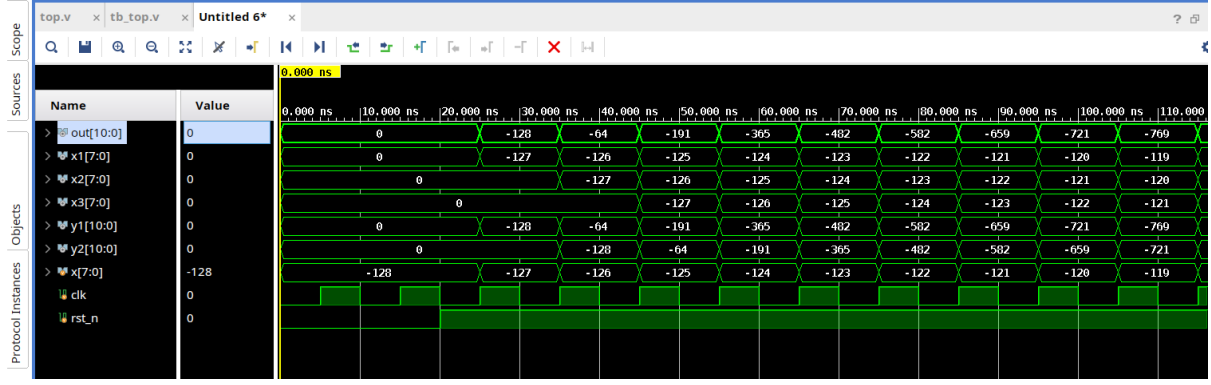


Figura 5: Diagrama de onda, filtro IIR

$$X_{min} = -2^{b-1}$$

Siendo $b = 8 \text{ bits}$.

$$X_{max} = 2^{8-1} - 1 = 127$$

$$X_{max} = 127$$

$$X_{min} = -2^{b-1} = -128$$

$$X_{min} = -128$$

Se realiza el análisis de crecimiento de bits de la salida respecto de los valores de entrada.

5.0.1. Camino directo (términos de entrada)

Se considera el valor máximo positivo para la suma de términos. Esto es:

$x[n]$	$x[n-1]$	$x[n-2]$	$x[n-3]$
X_{max}	X_{min}	X_{max}	X_{max}

Entonces el valor máximo positivo para la suma de términos de entrada será 23.

$$Max \text{ value of addition} = 4X_{max} = 4(2^{b-1} - 1) = 4(2^{8-1} - 1) = 508$$

Por lo que, en primer instancia, se necesitan 10 bits.

Se consideran los valores mínimos para la entrada.

$x[n]$	$x[n-1]$	$x[n-2]$	$x[n-3]$
X_{min}	X_{max}	X_{min}	X_{min}

Entonces la magnitud de la suma máxima negativa será:

$$Min \text{ value of addition} = 4X_{min} = 4(2^{b-1}) = -4(2^{8-1}) = -512$$

Se necesitan 10 bits para representar el valor mínimo.

La cantidad de bits necesaria para representar el rango de números de -512 a 508 es:

$$n = 10 \text{ bits}$$

5.0.2. Camino de realimentación

En el camino de realimentación se tienen las siguientes operaciones:

$$0,5y[n-1] + 0,25y[n-2]$$

Se debe tener en cuenta que los valores de y con sus respectivos delay son multiplicados por coeficientes menores a 1 ($coef < 1$). Lo cual reduce el valor y por lo tanto se reduce la cantidad de bits.

Para máxima resolución se considera el peor caso. Esto último sucede cuando la salida anterior se encuentra en su valor máximo. Si la salida es Y_{max} entonces se tiene:

$$0,5Y_{max} + 0,25Y_{max} = 0,75Y_{max}$$

5.0.3. Cálculo del peor caso total

Se plantea el valor máximo absoluto:

$$Y_{max} = 512 + 0,75Y_{max}$$

$$Y_{max}(1 - 0,75) = 512$$

$$0,25Y_{max} = 512$$

$$Y_{max} = 2048$$

Entonces la cantidad necesaria de bits para representar la salida son los siguientes:

$$2048 = 2^{11}$$

$cantidad\ de\ bits = 12bits$

Se necesitan 12 bits para representar -2048.

Rango de salida de $[-2048, +2047]$.

Se obtienen los siguientes resultados:

Entrada		Salida	
Valor mínimo	Valor máximo	Valor mínimo	Valor máximo
-128	-127	-2048	+2047

6. LABORATORIO 2 - FEC - Grupo 1

6.1. Ejercicio 1 - Matrices y parámetros

Escribir $\mathbf{G}$ y $\mathbf{H}$ a partir de $\mathbf{B}$ y verificar que $\mathbf{GH}^T = 0$ y $B^2 = 1$. Indicar dimensión, tasa y **overhead**. Codificar a mano $m = 0xA5C$ y comprobar que la palabra obtenida cumple $\mathbf{vH}^T = 0$.

6.2. Ejercicio 2 - Distancia mínima y capacidad de corrección

Determinar la distancia mínima, la capacidad de corrección t y la capacidad de detección a partir de la distribución de pesos del código:

La distancia mínima es:

$$d_{min} = 8$$

La capacidad de corrección del código es:

$$t = \frac{d_{min} - 1}{2}$$

$$t = \frac{8 - 1}{2} = \frac{7}{2} = 3,5 \approx 3$$

$$t = 3$$

La capacidad de detección:

$$capac. \text{ deteccion} = d_{min} - 1$$

$$capac. \text{ deteccion} = 8 - 1 = 7$$

$$capac. \text{ deteccion} = 7$$

La capacidad de detección está determinada por la distancia mínima menos uno debido a que si nuestra codeword se modificó en, por ejemplo 7 bits, entonces se tiene que el resultado no pertenece a una codeword.